

A calm reader, sharpened.

A review of the Maakdown v1 spec, the shipped reader UI, and the P8–P10 roadmap — with a working redesign you can click through. Verdict: the product thinking is disciplined and the bones are right. The biggest wins are in *functional aesthetics* — the chrome, not the engine.

SCOPE

Spec v0.3 · Roadmap P8–P10 · Shipped UI

LENS

UX · functional aesthetics · spec & sequencing

SYSTEM

Maakdown DS (paper & ink)

COMPANION

[Interactive prototype →](#)

00

The short version

Maakdown knows what it is: a fast, local-first **reader**, not an editor. The scope discipline (explicit “Out” list), the security model, and the virtualization-aware navigation are genuinely well-reasoned. What holds the current build back from feeling finished is the **chrome**: a toolbar of text labels with no hierarchy, engine internals leaking into the product surface, and raw data dumped into the metadata panel. None of that is hard to fix, and your own design system already ships the pieces to fix it.

✓ KEEP DOING

- Ruthless scope: viewer-only, explicit deferrals (transclusion, tags, mobile).
- Security-first asset model — tokenized loopback server over raw file URLs.
- Navigation that owns scrolling (anchor stabilization) instead of trusting the DOM.
- Per-phase exit criteria and a real perf harness with a Linux floor.

🔑 FIX FIRST

- Replace the text-label toolbar with a compact icon toolbar + tooltips.
- Pull the highlighter-engine picker out of the product chrome.
- Format dates & values in the metadata panel — no raw `Date.toString()`.
- Make a command palette the spine, not a P9 afterthought.
- Promote accessibility & settings from “P10/P8” to cross-cutting.



Tabs · ⌘K palette · ⌘F find · typography controls · focus mode · Mermaid zoom · light + dark.

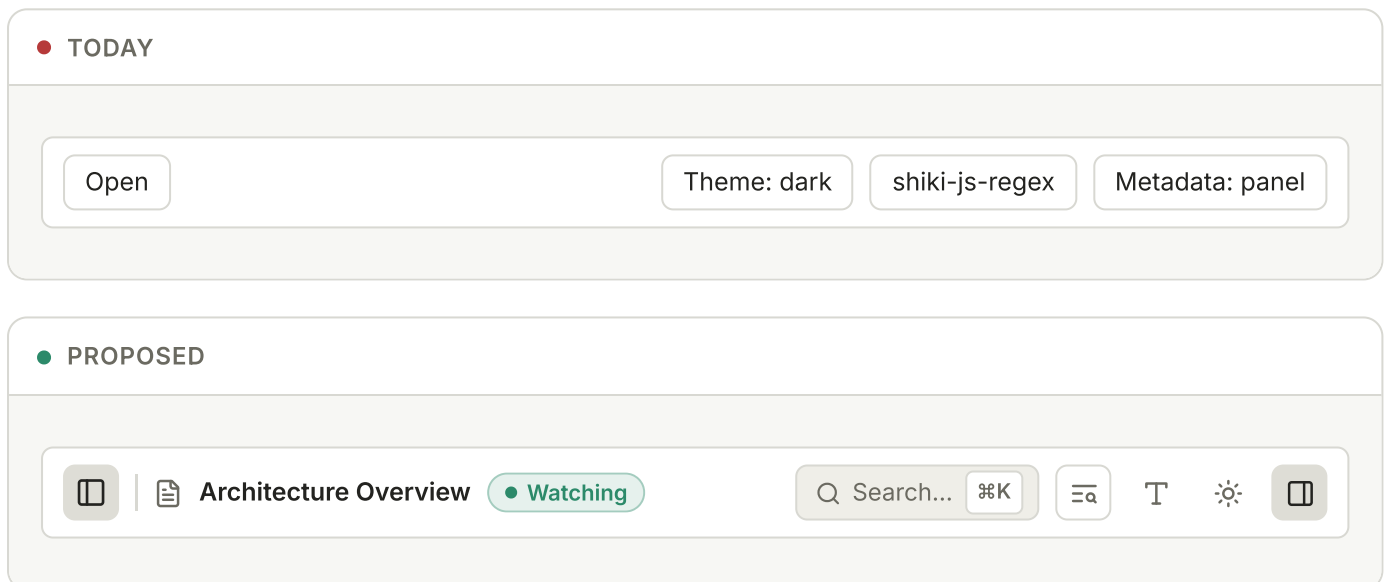
The shipped UI — functional aesthetics

Read against the two screenshots. These are surface issues, but surface is where “calm and finished” is won or lost in a reading tool.

The toolbar is doing too little with too much

Today the toolbar is four text pills — `Open`, `Theme: dark`, `shiki-js-regex`, `Metadata: panel` — all styled identically. Three problems: an **action** (`Open`) is visually indistinguishable from three **state read-outs**; the state labels change width as state changes, so the bar visibly jitters; and `shiki-js-regex` exposes a benchmark-only engine internal as a top-level control. Your DS already ships `IconButton` + Lucide for exactly this.

Toolbar — before / after



- **Three zones, one rule.** Left = navigation + document identity; right = view controls. State lives in the icon's pressed state and its tooltip, not in shifting text.
- **One search affordance.** A single `⌘K` field replaces guessing which pill does what — and absorbs `Open`, recents, headings and settings.
- **Engine choice leaves the chrome.** Ship `highlight.js` as the default; move the Shiki toggle behind Settings / a dev flag, per your own spec (§9 calls Shiki an *evaluation* engine).

The metadata panel is leaking raw data

The `generated` field reads `Thu Jun 04 2026 17:00:00 GMT-0700 (Pacific Daylight Time)` — a raw `Date.toString()`. It's the clearest “unfinished” tell in the whole UI. A formatting layer fixes this everywhere (dates, sizes, counts) and is cheap.

● TODAY

GENERATED

Thu Jun 04 2026 17:00:00 GMT-0700 (Pacific Daylight Time)

● PROPOSED

GENERATED

Jun 4, 2026 · 5:00 PM (local)

Section headings shouldn't look like web links

In the screenshots the headers (“Cross-service dependency model”) render as large, bright-blue, always-underlined links. That reads as a hyperlinked web page, not a calm document — and it fights the single-accent restraint your color tokens are built around. Render headings as ink, and reveal a quiet anchor (#) affordance on hover for the copy-link action.



READING MEASURE

Cap prose, let diagrams breathe

Spec §5 commits to an ~860px measure, but the screenshots show full-width content. Keep prose capped for legibility while letting Mermaid, wide tables, and code escape the measure. The prototype does this per-block.



SIDEBAR IDENTITY

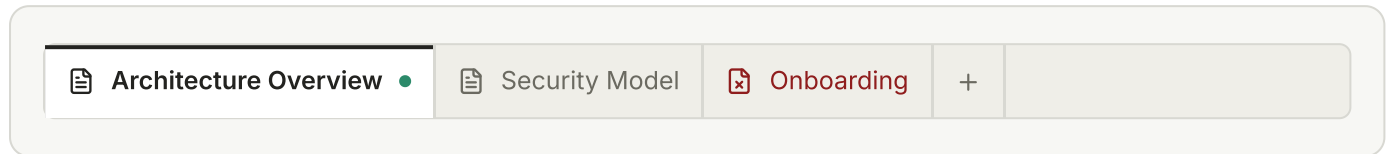
Separate “what doc” from “outline”

Today the document title doubles as the top of the TOC with no app wordmark and no labelled “Outline” group. A small wordmark + an uppercase “Outline” label restores hierarchy (your DS Sidebar already models this).

The redesigned shell

One recommended direction — faithful to the paper-and-ink language, just sharper — plus where I'd let it get a little more opinionated. All of this is live in the prototype.

Tab strip (P8) — document identity, watching state, error tabs



The recommendation (faithful)

- **Compact icon toolbar** in three zones, with a persistent `⌘K` field as the product's spine.
- **Tab strip** with file-text glyph, a small green "watching" dot, an error glyph + red title for missing files, and a scrollable overflow.
- **Wordmark sidebar** → labelled *Outline* with depth indents and a clear active-row fill from `--active`.
- **Measured reader**, refined metadata panel (formatted values, status badge, tag chips), and quiet hairline separators throughout — no new shadows.

Where it could get more opinionated

- **Reading display as a first-class control** (typeface, size, line-height, measure) — not buried. The prototype exposes it as an `Aa` popover that applies live.
- **Focus mode** that dissolves all chrome to a single centered column, keeping only Find and an Exit affordance — the strongest expression of "the chrome should disappear."
- **A serif reading option.** Use a system serif stack (Georgia / lowan Old Style) so there's no extra download; it makes long-form notes feel like a page, not an app.

✦ Try it

In the prototype: `⌘K` for the palette, `⌘F` to find (with match count + case/word), the `Aa` button for typography, `F` for focus mode, and click the pipeline diagram to open the zoom inspector. Toggle dark from the toolbar.

The P8–P10 roadmap

The three-phase shape is right: workspace → productivity → editorial polish. Notes on each, with the sequencing risks called out.

☐ P8

Desktop Workspace



STRONG

Per-tab state isolation & session restore

Independent model/scroll/watcher per tab, reuse-by-canonical-path, and recoverable error tabs for missing files are exactly the right instincts. Versioned, atomically-written JSON is the correct persistence call.



RISK · SEQUENCING

The multi-path watcher is the tall pole

Generalising the watcher to N paths plus safe-save coalescing is the riskiest P8 item. De-risk by shipping tabs that watch only the *active* tab first, then widen to all open paths — it's a smaller, testable step toward the same end state.



GAP

Define a few tab edge-cases explicitly

What does dropping a *folder* do (open vault) vs a *file* (open tab)? Unresolved wikilinks must **not** open a tab — show inline. On session restore, persisted docs must **re-resolve assets** (tokens rotate per process, per §10) rather than reuse stale loopback URLs.



RISK · THE TRAP

Search count must be full-document, highlights lazy

"Only mounted blocks receive `<mark>`" is correct — but the `3/12` counter must be computed over the full document model's plain text, not the mounted subset, or the number lies. Next/Prev must drive the virtualizer to materialise the target block — reuse the P5 anchor-stabilisation machinery. The prototype models the count + current + case/whole-word behaviour.



PROMOTE

The command palette should arrive earlier

It's listed inside P9, but it's the cheapest way to *relieve toolbar pressure* across every phase — commands, open tabs, recents, headings, settings. Treat it as the navigation spine and build a thin version in P8.



RISK · MEMORY

Print/PDF fights virtualization

"Expand virtualized content before printing, then restore" is the other tall pole: fully materialising a 10k-line doc plus forcing all KaTeX/Mermaid/highlight to render can spike RSS. It's acceptable for a transient, user-initiated action — but gate it (progress state, a force-enhance-all path, and a hard restore on completion/cancel).



STRONG

This is the right home for functional aesthetics

Compact icon toolbar, resizable/collapsible panels, document-centered canvas, typography controls, focus mode, responsive drawers — all endorsed. Most of Part 01's fixes live here; just pull the toolbar + formatting forward so the product doesn't look unfinished for three phases.



GAP · CROSS-CUTTING

Accessibility can't be a final-phase bolt-on

Focus ring, reduced-motion, ARIA on existing controls, and focus-trapping for dialogs/palette should be requirements from P1, not collected in P10. The hard parts (live-region announcements for load/reload/search/copy) can land in P10, but the baseline is cheap to keep green continuously.



GAP

There's a hidden "Settings" surface

Theme, highlighter, metadata mode, and now four typography settings are accumulating with no home. The P8 ConfigService → persistent settings service should own them all behind one surface, reachable from the palette — instead of each becoming another toolbar pill.

Spec & technical notes

The spec is unusually disciplined. These are gaps and risks, not objections.



SOUND

Wails v2 + loopback asset server

Choosing the stable Wails v2 line and routing local images through a tokenized `127.0.0.1` server (rather than the v2 dynamic asset handler that breaks with Vite 5+) is well-reasoned and keeps dev/prod identical. No objection — just make session restore re-resolve assets, since tokens rotate per process.



GAP · POLISH

No formatting / locale layer

The raw date in the metadata panel is the symptom. Define one formatting module (dates, file sizes, counts, durations) so every surface — metadata, perf harness output, error messages — reads consistently. Add a locale snapshot test to catch regressions.



GAP

Error taxonomy is implied, not enumerated

The spec names a couple ("Could not open document", blocked assets). Enumerate the full set — missing file, permission denied, too large, unsupported type, parse failure, watcher lost — and pin each to a defined reader presentation (P10 already commits to broken-asset and unresolved-wikilink states; extend that list).



RISK · PERF

Re-run the harness after P8 and P9, not just at release

Tabs and search both touch the hot path. Your test plan already says "run release-check after every phase" — hold to it, and add the "10k-line fixture with several open tabs, only active mounts" scenario as a standing benchmark.



DECISION

Decide the highlighter story for v1

Shiki is described as an evaluation engine, yet `shiki-js-regex` ships as a visible control. Pick one: ship highlight.js and gate the Shiki toggle behind a dev/eval flag. Don't surface engine internals as product chrome.

If I had your backlog

Highest leverage first. "Impact" is on the felt quality of the product; "Effort" is rough.

MOVE	WHEN	IMPACT	EFFORT	WHY
Icon toolbar + tooltips	Now	high	low	Single biggest "finished" gain; the DS already ships IconButton.
Formatting / locale layer	Now	med	low	Kills the raw-date tell; one module pays off everywhere.
Pull engine picker from chrome	Now	med	low	Removes a confusing internal from the product surface.
A11y baseline (ring, ARIA, reduced-motion)	Now	med	low	Cheap to keep green; expensive to retrofit in P10.
Command palette (thin)	Next	high	med	The spine; relieves toolbar pressure across all phases.
Tabs watching active-only first	Next	high	med	Ships the workspace before the multi-path watcher refactor.
Reading display controls	Next	high	med	Defining feature for a reader; persist globally.
Full-doc search + lazy highlight	Next	high	high	Reuse P5 anchor stabilisation; get the count right.
Focus mode	Later	med	low	The purest expression of the product thesis.

MOVE	WHEN	IMPACT	EFFORT	WHY
Print/PDF with guarded expansion	Later	med	high	Valuable but memory-sensitive; gate it carefully.

Most of “Now” and “Next” is already demonstrated there.

Review built in the Maakdown design system — warm paper, ink, hairline borders, a single ink-blue accent. Mockups and prototype use the system’s real tokens and components. The pipeline diagram in the prototype is a placeholder for a live Mermaid render.